

Smart Lab Management System (SLMS)

Software Requirements Specification (SRS) and Complete Project Documentation

Final Year Diploma Project Documentation

Version 1.0

Prepared for: Project Mentor / Department of Computer Engineering

Prepared by: Project Team

Academic Year: 2026-2027

Purpose of this document: A single readable document that explains the complete project concept, requirements, architecture, modules, database, APIs, implementation plan, testing, results and future enhancements.

Document Control

Item	Details
Project Title	Smart Lab Management System (SLMS)
Project Type	LAN-based desktop/web hybrid monitoring and remote management system
Document Type	SRS + Project Documentation
Version	1.0
Status	Working project document for mentor review and project development
Prepared For	Computer Engineering Department / Project Mentor
Prepared By	Project Team
Core Environment	College computer laboratory connected through LAN

Revision History

Version	Date	Description	Author
1.0	July 2026	Full project documentation prepared from the SLMS master structure	Project Team

Abstract

Smart Lab Management System (SLMS) is a client-server based software system designed for college computer laboratories. The system helps an administrator monitor all lab computers from a central host computer. Each client computer runs a lightweight Python client agent that collects system information such as CPU usage, RAM usage, disk usage, IP address, operating system details, installed applications, running applications, online/offline status and reported problems. The collected information is sent to the host server through REST API and WebSocket communication over the local network.

The administrator can view the live status of all computers through a React-based dashboard, identify problems quickly, maintain issue records, track application usage, generate reports and send approved remote commands such as message, lock, restart or shutdown. The project also includes a future-ready AI-assisted module for issue classification, anomaly detection and maintenance recommendation. The system is designed to work in a college lab LAN without depending on continuous internet connectivity.

The objective of SLMS is to reduce manual lab inspection, improve fault visibility, maintain proper computer records and support secure remote management. The system uses Python, FastAPI, Uvicorn, SQLAlchemy, SQLite/PostgreSQL, React, Vite, WebSocket, Tkinter, PyInstaller and Electron in a modular architecture.

Keywords

- Smart Lab Management System
- Computer lab monitoring
- Remote management
- FastAPI
- WebSocket
- Python client agent
- React dashboard
- LAN-based system
- Software inventory
- Maintenance tracking

Table of Contents

1. Introduction
 2. Existing System and Literature Survey
 3. Problem Statement, Objectives and Scope
 4. Requirement Analysis
 5. System Architecture
 6. Technology Selection and Technology Stack
 7. Module Design
 8. Database Design
 9. API Documentation
 10. User Interface Design
 11. UML and Behavioral Design
 12. Algorithms and Core Logic
 13. Implementation Plan
 14. Testing and Validation
 15. Security, Privacy and Reliability
 16. Deployment and User Manual
 17. Results and Expected Output
 18. Limitations and Future Enhancements
 19. Conclusion
 20. References
- Appendices

Chapter 1: Introduction

1.1 Project Overview

Smart Lab Management System is a software application for monitoring and managing computers in a college laboratory. In a normal lab, teachers or lab assistants must manually check whether each system is working, which applications are installed, which computers have storage or performance issues and which systems require maintenance. SLMS solves this problem by creating a central monitoring system where all client computers report their status to a host server.

The project follows a host-client architecture. The host computer runs the server, database and administrator dashboard. Each lab computer runs a client agent. The client agent collects system details and sends them to the host. The administrator can observe all systems from one place, detect issues early and perform permitted remote operations.

1.2 Need for the Project

- Manual checking of each lab computer consumes time and effort.
- Faults are often noticed only when a student or teacher reports them.
- There is usually no proper maintenance history for each computer.
- Installed software and running applications are difficult to track manually.
- Administrators need a centralized view of system health, usage and alerts.
- A LAN-based system is suitable because most lab computers are already connected in the same network.

1.3 Project Vision

The vision of SLMS is to provide a centralized, secure and easy-to-use platform for lab administrators to monitor, manage and maintain computer systems. The system should help the lab move from manual inspection to automated monitoring and planned maintenance.

1.4 Project Users

User Role	Description	Main Responsibilities
Administrator	Main user who controls the system	Monitor systems, view reports, send commands, manage users and settings
Lab Assistant / Technician	Maintenance user	View issues, update maintenance records, fix reported problems
Teacher / Faculty	Observer or limited user	Check lab readiness, view computer availability and issue status
Student	Indirect user	Uses client systems; may report problems through the client interface if enabled

1.5 Exclusion

QR codes are not part of this project. Client computers are identified automatically using device ID, hostname, IP address, network adapter/MAC information, lab number and system number.

Chapter 2: Existing System and Literature Survey

2.1 Existing Manual System

In many computer laboratories, lab status is maintained manually. The lab assistant checks systems one by one, records issues in registers and installs or verifies software manually. This approach may work for a small number of systems but becomes inefficient when the number of computers increases.

Existing Practice	Limitation
Manual checking of CPU/RAM/disk status	Slow and not continuous
Manual software verification	Difficult to know which system has missing or unwanted software
Issue register or verbal reporting	Problem history may be incomplete or lost
Physical inspection for every problem	Consumes staff time
No live dashboard	Administrator cannot immediately know system status

2.2 Related Systems

There are many commercial remote monitoring and management tools. However, such tools may require paid licenses, internet/cloud accounts or complex setup. A college project version can focus on local network monitoring, basic remote management, database records, issue tracking and a simple administrator dashboard.

2.3 Learning from Related Concepts

- **Network monitoring:** Continuous status collection helps identify offline systems and resource problems.
- **Asset management:** Hardware and software inventory helps the lab maintain proper computer records.
- **Remote administration:** Controlled remote commands reduce the need for physical access for basic actions.
- **Issue tracking:** Problem reports, priorities and maintenance logs improve accountability.
- **Analytics and AI assistance:** Repeated issues and unusual usage patterns can be converted into useful recommendations.

2.4 Proposed Improvement

The proposed system combines monitoring, inventory, issue reporting, maintenance records, reporting and approved remote management into one modular platform designed specifically for a college lab environment.

Chapter 3: Problem Statement, Objectives and Scope

3.1 Problem Statement

College computer laboratories need a centralized system to monitor client computers, track system health, identify software and hardware issues, maintain problem history and perform secure remote management. Manual monitoring is time-consuming and does not provide real-time visibility. Therefore, SLMS is proposed as a LAN-based host-client system for efficient lab monitoring and management.

3.2 Objectives

- To automatically collect computer details from each client system.
- To show live CPU, RAM, disk and online/offline status in the administrator dashboard.
- To maintain records of installed applications, running applications and usage history.
- To allow users or technicians to report problems and track issue status.
- To generate alerts for low storage, high resource usage or inactive systems.
- To allow approved remote commands from host to client systems.
- To maintain audit logs for commands and important activities.
- To support future AI-assisted diagnostics and maintenance recommendations.

3.3 Scope of the Project

In Scope	Out of Scope
LAN-based host-client communication	Public internet-based remote access as a mandatory feature
Computer monitoring and inventory	Full enterprise device management platform
Issue reporting and maintenance records	Automatic hardware repair
Approved remote commands	Unrestricted remote control of every system operation
Reports and analytics	Advanced commercial-level business intelligence
Future-ready AI module	AI auto-executing commands without administrator approval
Automatic client identification	QR-code based computer identification

3.4 Success Criteria

- The host must show connected clients and their latest status.
- The client must send system information reliably over the LAN.
- The dashboard must show live metrics and issue records clearly.
- The database must store computer, software, usage, issue and command data.
- Remote commands must be permission-based and logged.
- The system must continue working inside the lab LAN without mandatory internet.

Chapter 4: Requirement Analysis

4.1 Functional Requirements

FR ID	Requirement	Description
FR-01	Client registration	Each client computer must register automatically with the server using hostname, IP, OS and device identifiers.
FR-02	System monitoring	Client must collect CPU usage, RAM usage, disk usage, OS details, boot time and network details.
FR-03	Heartbeat	Client must send periodic status updates so the server can detect online/offline state.
FR-04	Software inventory	Client must collect installed application details where supported by the operating system.
FR-05	Running applications	Client must detect active/running applications and avoid unnecessary duplicate system-process noise.
FR-06	Issue reporting	Users or technicians must be able to report problems against a computer.
FR-07	Maintenance records	Technicians must update repair details, status and remarks.
FR-08	Dashboard	Administrator must view system cards, live metrics, alerts, issues and reports.
FR-09	Remote commands	Administrator must send approved commands such as message, lock, restart or shutdown.
FR-10	Audit log	System must log important actions, commands and user activity.
FR-11	Notifications	System must show alerts for low disk, high resource usage, offline systems or unresolved issues.
FR-12	Reports	System must generate summaries of computer status, issues, usage and maintenance.

4.2 Non-Functional Requirements

Category	Requirement
Performance	Client updates should be lightweight and should not affect normal lab usage.
Reliability	Server should handle temporary client disconnection and mark systems offline after timeout.
Security	Remote commands must require authentication, authorization and audit logging.
Usability	Dashboard should be clear enough for teachers and lab assistants to understand quickly.

Maintainability	System should use modular backend, client and frontend structure.
Scalability	Design should support a small lab initially and multiple labs in future.
Portability	Client should be packaged using PyInstaller; host dashboard can be packaged using Electron.
Availability	Core system should work in LAN without mandatory cloud dependency.

4.3 Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Host computer	Dual-core CPU, 4 GB RAM, 10 GB free disk	i3/i5 or better, 8 GB RAM, SSD
Client computer	Windows/Linux PC with LAN connectivity	Windows lab PCs with stable LAN connection
Network	LAN switch/router	Stable wired LAN for lab systems
Display	Basic monitor for admin dashboard	Large monitor for dashboard view

4.4 Software Requirements

Layer	Software / Tool	Purpose
Client	Python	Client agent development
Client	psutil, platform, socket	System monitoring and identification
Client	Windows API	Windows-specific lock/shutdown/app information where needed
Client	Tkinter	Small client interface and status display
Client	PyInstaller	Packaging Python client as executable
Backend	FastAPI, Uvicorn	REST API, WebSocket server and application server
Backend	SQLAlchemy	Database access layer
Database	SQLite, PostgreSQL	Prototype database and final production database
Frontend	React, Vite	Admin dashboard
Desktop	Electron	Host dashboard desktop packaging
Tools	Git, GitHub, Postman, pgAdmin	Version control, API testing and database management

Chapter 5: System Architecture

5.1 Architecture Style

SLMS follows a modular client-server architecture. The client agent runs on each lab computer. The monitoring server runs on the host computer. The database stores all persistent records. The administrator console displays the data and allows controlled operations. Communication between client and server happens through REST API and WebSocket over LAN.

Smart Lab Management System - High Level Architecture

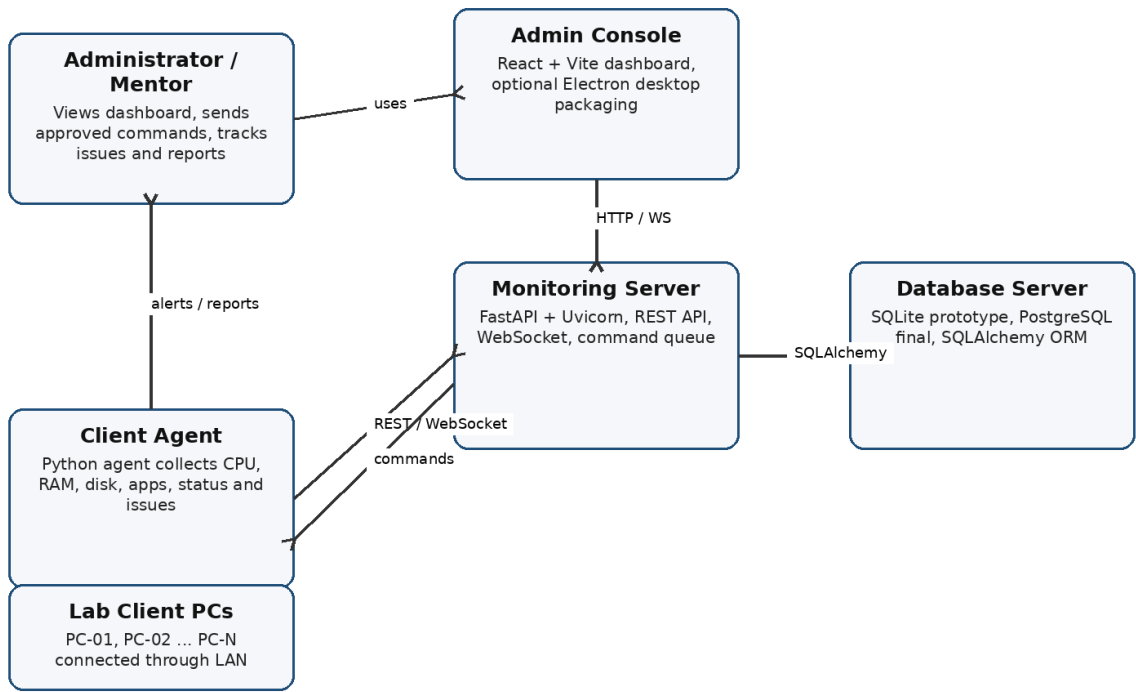


Figure 5.1: High level architecture of Smart Lab Management System

5.2 Logical Components

Component	Responsibility
Client Agent	Collects system data, sends heartbeat, receives approved commands and reports local status.
Monitoring Server	Receives data, validates requests, stores records, broadcasts live updates and manages command queue.
Database Server	Stores computers, metrics, software, usage history, issues, commands, users and audit logs.
Administrator Console	Provides dashboard, reports, alerts, issue tracking and command interface.
AI Assistance Layer	Analyzes repeated problems, abnormal usage and maintenance patterns for recommendations.

5.3 Data Flow

System Data Flow

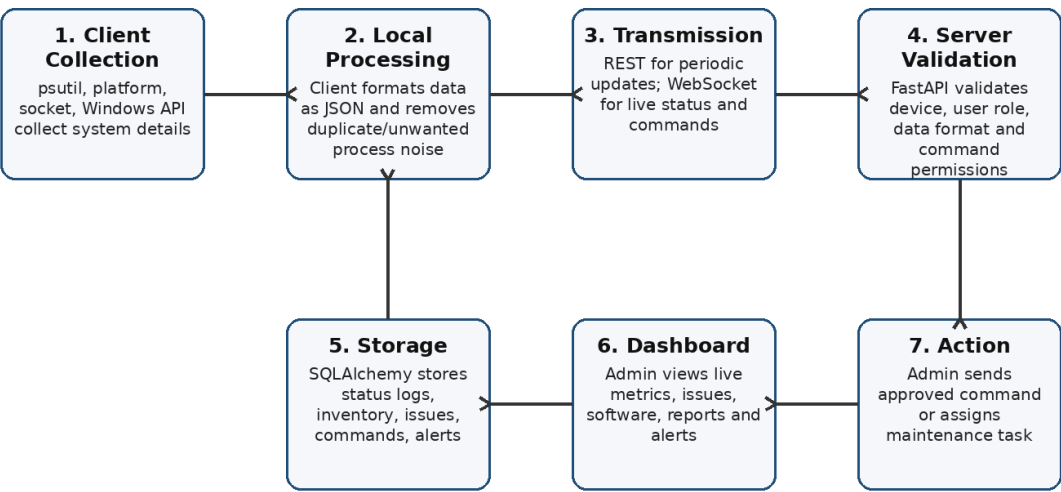


Figure 5.2: Data flow from client collection to administrator action

5.4 Deployment View

LAN Deployment View

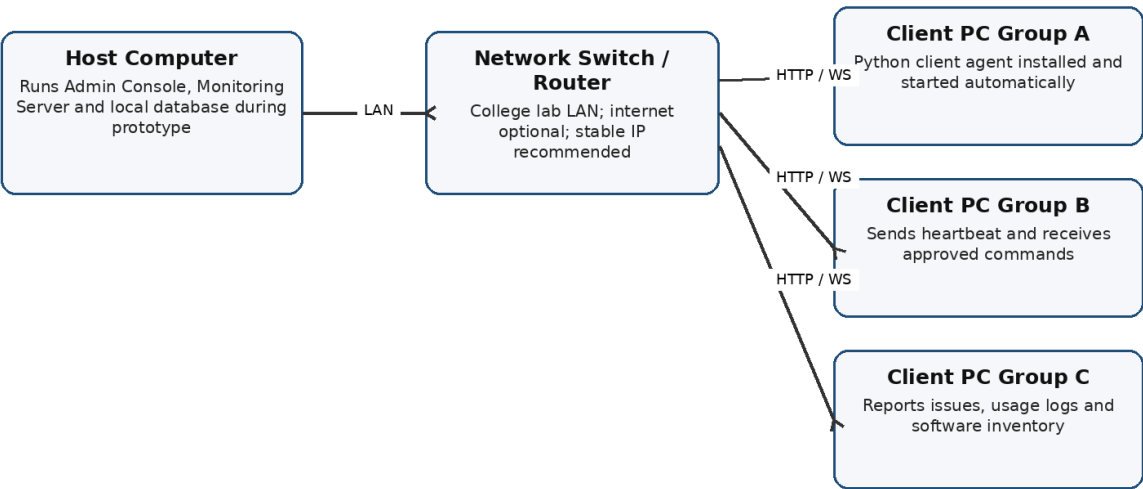


Figure 5.3: LAN deployment view

5.5 Communication Methods

Communication Type	Used For	Reason
--------------------	----------	--------

REST API	Registration, periodic status upload, issue creation, software inventory upload	Simple request-response communication and easy testing through API docs/Postman
WebSocket	Live metrics, online status, alerts and remote command delivery	Real-time two-way communication without repeatedly refreshing dashboard
Database ORM	Backend to database operations	SQLAlchemy keeps database operations organized and portable

Chapter 6: Technology Selection and Technology Stack

6.1 Selected Technology Stack

Part	Selected Technology	Use in Project
Client Language	Python	Build the monitoring agent that runs on every lab computer.
Monitoring Libraries	psutil, platform, socket	Collect CPU, RAM, disk, OS, hostname, IP and network details.
Windows Operations	Windows API where necessary	Perform Windows-specific operations such as lock or safe shutdown commands.
Communication	REST API and WebSocket	Send periodic data and receive live updates/commands.
Client Interface	Tkinter	Simple local client window for status and issue reporting.
Client Packaging	PyInstaller	Convert Python client into executable for installation on lab PCs.
Backend Language	Python	Keep backend and client in the same language family.
Backend Framework	FastAPI	Build REST endpoints and WebSocket support.
Server	Uvicorn	Run the FastAPI application.
Database Access	SQLAlchemy	Define models and interact with SQLite/PostgreSQL.
Prototype DB	SQLite	Fast local development and testing.
Final DB	PostgreSQL	Stable multi-user database for final deployment.
Frontend	React	Build dynamic administrator dashboard.
Build Tool	Vite	Fast frontend development server and build process.
Host Packaging	Electron	Package admin dashboard as desktop application if needed.

6.2 Why This Stack Was Selected

Technology	Reason for Selection
Python	Simple syntax, strong system-monitoring libraries and suitable for student project development.
psutil	Directly supports CPU, memory, disk, process and network statistics.
FastAPI	Modern Python framework with automatic API documentation, validation and WebSocket support.
Uvicorn	ASGI server required to run FastAPI efficiently.
SQLAlchemy	Reduces raw SQL repetition and supports migration from SQLite to PostgreSQL.

SQLite	No separate database server needed during prototype.
PostgreSQL	Better for final system with more records, users and reports.
React	Suitable for building reusable dashboard components and live UI updates.
Vite	Fast React project setup and development experience.
Electron	Allows the host dashboard to behave like a desktop application.
Tkinter	Included with Python, useful for a lightweight client-side interface.
PyInstaller	Helps deploy the client without asking users to run Python source code manually.

6.3 Alternatives Considered

Alternative	Reason Not Prioritized
Node.js backend	Backend is kept in Python to match the client agent and reduce learning load.
Only browser dashboard	Browser dashboard is good for testing, but final host may be packaged as desktop app.
Cloud-only deployment	Not required because the college lab can run fully over LAN.
QR-code identification	Removed from project; automatic device identification is simpler and more suitable.
Heavy commercial RMM tools	May require licenses, internet dependency and features beyond project scope.

Chapter 7: Module Design

7.1 Module List

Module ID	Module Name	Main Responsibility
M-01	Client Monitoring Agent	Collect client computer details and send them to host.
M-02	Client Registration and Identification	Identify each computer automatically and maintain unique records.
M-03	Host Backend API	Receive, validate, process and store data.
M-04	Real-Time Communication	Provide WebSocket-based live dashboard and command delivery.
M-05	Admin Dashboard	Show all client computers, alerts, issues, reports and controls.
M-06	Software Inventory	Track installed applications and running applications.
M-07	Usage History	Record application usage and active process information.
M-08	Problem Reporting	Allow issue creation, tracking and resolution records.
M-09	Maintenance Management	Store repair history and technician remarks.
M-10	Remote Command Management	Send approved commands and record command results.
M-11	Alerts and Notifications	Notify high usage, low disk, offline systems and unresolved issues.
M-12	Analytics and Reports	Generate lab status, issue, maintenance and usage reports.
M-13	AI Assistance Module	Provide diagnosis support, anomaly detection and recommendations.
M-14	Security and Audit Log	Track users, permissions, command logs and important events.

7.2 Client Monitoring Agent

The client agent is installed on each lab computer. It starts when the system starts, collects data at fixed intervals and sends the data to the monitoring server. It should be lightweight and should not interrupt students while they use the system.

- Collects system details: hostname, IP, OS, CPU, RAM, disk and boot time.
- Collects software inventory and running application details.
- Sends heartbeat to server to indicate online status.
- Receives command messages through WebSocket.
- Shows optional Tkinter status window for issue reporting or connection status.

7.3 Host Backend Module

The backend is the central brain of SLMS. It accepts data from clients, stores data in the database, exposes APIs to the frontend, validates users and handles real-time communication.

- REST API for registration, metrics upload, issue management and reports.
- WebSocket for live updates and command channel.
- Database models using SQLAlchemy.
- Business rules for alerts, command permissions and data validation.

7.4 Admin Dashboard Module

The dashboard is used by the administrator to understand the complete status of the lab. It should provide system cards, filters, charts, issue tables, command buttons and report export options.

7.5 Remote Command Module

Remote commands must be restricted, authenticated and logged. The system should not allow arbitrary commands. Only approved operations such as send message, lock system, restart system, shutdown system and request refresh should be available. Every command must have command status such as pending, delivered, executed, failed or cancelled.

7.6 AI Assistance Module

The AI layer is not the first requirement for the prototype. In the initial version, rule-based diagnostics can be used. Later, the system can analyze issue history, repeated failures, high usage patterns and maintenance records to suggest probable causes and maintenance actions.

Chapter 8: Database Design

8.1 Database Overview

The database stores all information collected from client computers and all actions performed by administrators. SQLite is used during development because it is simple and does not require a separate server. PostgreSQL is used for the final system because it is better for multiple users, larger datasets and stable deployment.

8.2 Main Tables

Table Name	Purpose
users	Stores administrator, technician and faculty login details.
roles	Stores role names and permission categories.
computers	Stores each lab computer identity and basic system information.
client_status	Stores latest online/offline status and heartbeat timestamp.
system_metrics	Stores CPU, RAM, disk and network usage records.
hardware_information	Stores processor, memory, disk and OS details.
software_inventory	Stores installed applications and versions.
running_processes	Stores currently running applications/processes.
application_usage	Stores application usage start time, end time and duration.
issue_reports	Stores reported problems and their status.
maintenance_records	Stores repair actions and technician notes.
remote_commands	Stores commands sent from host to client.
command_results	Stores result of executed commands.
notifications	Stores alerts and messages shown to users.
audit_logs	Stores important user actions and system events.
ai_recommendations	Stores AI/rule-based suggestions for issues and maintenance.

8.3 Important Table Structures

Table	Important Fields
computers	computer_id, lab_no, system_no, hostname, ip_address, mac_address, os_name, os_version, status, registered_at
system_metrics	metric_id, computer_id, cpu_usage, ram_usage, disk_usage, network_sent, network_received, recorded_at
software_inventory	software_id, computer_id, software_name, version, publisher, installed_date, detected_at
application_usage	usage_id, computer_id, application_name, start_time, end_time, duration_seconds
issue_reports	issue_id, computer_id, reported_by, category, description, priority, status, created_at, resolved_at
remote_commands	command_id, computer_id, command_type, command_payload, issued_by, status, issued_at

audit_logs	log_id, user_id, action, target_type, target_id, ip_address, created_at
------------	---

8.4 Relationships

- One computer can have many system metric records.
- One computer can have many software inventory records.
- One computer can have many issue reports.
- One issue can have many maintenance records or comments.
- One administrator can issue many remote commands.
- Every remote command can have one or more command result entries.
- Audit logs are connected to users and important system events.

8.5 Sample SQLAlchemy Model

```
class Computer(Base):
    __tablename__ = "computers"

    id = Column(Integer, primary_key=True, index=True)
    hostname = Column(String, unique=True, nullable=False)
    ip_address = Column(String, nullable=False)
    os_name = Column(String)
    os_version = Column(String)
    status = Column(String, default="offline")
    registered_at = Column(DateTime, default=datetime.utcnow)
```

Chapter 9: API Documentation

9.1 API Overview

The backend exposes REST API endpoints for normal request-response operations and WebSocket endpoints for live communication. The API should use JSON format. Each request should be validated before storing data or performing actions.

9.2 REST API Endpoints

Method	Endpoint	Purpose
POST	/api/clients/register	Register a client computer with the server.
POST	/api/clients/heartbeat	Update online status and last seen time.
POST	/api/metrics	Upload CPU, RAM, disk and network metrics.
POST	/api/software/inventory	Upload installed software list.
POST	/api/processes/running	Upload running applications/processes.
GET	/api/computers	Get all registered computers.
GET	/api/computers/{id}	Get details of one computer.
GET	/api/computers/{id}/metrics	Get metric history of one computer.
POST	/api/issues	Create an issue report.
GET	/api/issues	List issue reports with filters.
PUT	/api/issues/{id}	Update issue status, priority or assignment.
POST	/api/commands	Create an approved remote command.
GET	/api/commands/{id}	Check command status and result.
GET	/api/reports/lab-status	Generate lab status summary.
GET	/api/reports/issues	Generate issue report summary.

9.3 WebSocket Endpoints

Endpoint	Connected By	Purpose
/ws/dashboard	Admin dashboard	Receive live computer status, metrics and alerts.
/ws/client/{computer_id}	Client agent	Receive remote commands and send live acknowledgements.

9.4 Sample JSON Payload

```
{
  "hostname": "LAB-PC-01",
  "ip_address": "192.168.1.25",
  "os_name": "Windows",
  "cpu_usage": 34.5,
  "ram_usage": 62.1,
  "disk_usage": 78.4,
  "recorded_at": "2026-07-07T10:30:00"
}
```

9.5 API Validation Rules

- Computer identity must be verified before accepting metrics.
- Only authenticated administrators can send remote commands.
- Command type must be selected from the approved command list.
- Invalid or incomplete JSON should return a clear error response.
- Every command request must be recorded in audit logs.

Chapter 10: User Interface Design

10.1 UI Goals

- Show complete lab status on one dashboard.
- Use simple cards and tables so non-technical users can understand quickly.
- Show warning indicators for high CPU, high RAM, low disk and offline systems.
- Provide filters by lab, status, issue priority and software.
- Keep remote command buttons clearly separated and protected by confirmation.

10.2 Main Screens

Screen	Purpose	Main Elements
Login	Authenticate admin/technician	Username, password, role-based access
Dashboard Home	Overall lab view	Total systems, online/offline count, alerts, recent issues
Computer List	View all computers	Computer cards/table, IP, OS, CPU, RAM, disk, last seen
Computer Detail	Deep view of one PC	Hardware info, metrics history, software, processes, command history
Software Inventory	Track installed software	Software name, version, publisher, installed date, computer list
Issue Management	Track problems	Issue category, priority, status, assigned user, remarks
Maintenance Records	Repair history	Technician, action taken, date, result
Reports	Generate summaries	Lab status, issue report, usage report, maintenance report
Settings	Configure thresholds and roles	CPU/RAM/disk thresholds, command permissions, lab details

10.3 Client Interface

The client interface can be a small Tkinter window showing connection status, hostname, IP address and a simple issue reporting option. The client should also work silently in the background when the window is closed or minimized, depending on project implementation.

10.4 Dashboard Wireframe Description

- Top navigation: project name, user profile, notification icon and logout.
- Summary cards: total computers, online computers, offline computers, active issues and critical alerts.
- Computer grid: each card shows computer name, IP, CPU, RAM, disk and status.
- Right panel: latest alerts, recent commands and unresolved issues.
- Bottom section: reports, usage charts and maintenance summary.

Chapter 11: UML and Behavioral Design

11.1 Use Case Design

Actor	Use Cases
Administrator	Login, view dashboard, view computers, send command, manage users, generate reports, configure thresholds
Technician	View assigned issues, update maintenance record, add repair notes, mark issue resolved
Client Agent	Register computer, send heartbeat, upload metrics, upload software inventory, receive command
Teacher / Faculty	View lab readiness, check issue status, report problem if allowed

11.2 Class / Entity Design

Entity / Class	Attributes	Relationships
User	id, name, username, password_hash, role_id	User creates commands, issues and maintenance records
Computer	id, hostname, ip, os, status, lab_no, system_no	Computer has metrics, software, issues and commands
SystemMetric	id, computer_id, cpu, ram, disk, recorded_at	Metric belongs to one computer
IssueReport	id, computer_id, category, priority, status	Issue belongs to one computer and may have maintenance record
RemoteCommand	id, computer_id, command_type, status, issued_by	Command belongs to one computer and one issuing user
Notification	id, type, message, target_user, created_at	Notification may be generated from metrics or issues

11.3 Sequence: Client Status Update

1. Client agent collects CPU, RAM, disk, OS and IP details.
2. Client formats the information as JSON.
3. Client sends the data to the FastAPI endpoint.
4. Backend validates the computer identity and payload.
5. Backend stores latest status and metric history in the database.
6. Backend broadcasts update to connected dashboard through WebSocket.
7. Dashboard updates the computer card without page refresh.

11.4 Sequence: Remote Command

1. Administrator logs in and selects a computer.
2. Administrator selects an approved command.
3. Backend verifies permission and stores command as pending.

4. Server sends command to the connected client through WebSocket.
5. Client checks command type and executes only whitelisted action.
6. Client sends result back to server.
7. Server updates command status and audit log.
8. Dashboard displays command result.

Chapter 12: Algorithms and Core Logic

12.1 Client Monitoring Algorithm

```
START
Load client configuration
Collect hostname, IP address, OS and hardware details
Register client with server if not already registered
LOOP every configured interval:
    Collect CPU, RAM and disk usage
    Collect running applications if enabled
    Send heartbeat and metrics to server
    Check WebSocket for pending command
    If command is valid and approved, execute permitted action
    Send command result to server
END LOOP
```

12.2 Server Heartbeat Algorithm

```
START
Receive heartbeat from client
Validate computer identity
Update last_seen timestamp
Set computer status = online
If metrics cross threshold:
    Create alert notification
Broadcast status update to dashboard
Periodically check computers:
    If current_time - last_seen > timeout:
        Set computer status = offline
        Create offline notification
END
```

12.3 Alert Generation Rules

Condition	Alert Type	Example Action
CPU usage above threshold for repeated intervals	High CPU alert	Show warning on dashboard
RAM usage above threshold	High RAM alert	Suggest closing heavy applications
Disk usage above threshold	Low storage alert	Mark system for cleanup
No heartbeat for timeout period	Offline alert	Notify lab assistant
Same issue repeated on same computer	Repeated fault alert	Recommend maintenance inspection

12.4 Issue Priority Logic

- Low priority: Minor issue that does not stop system usage.
- Medium priority: Issue affects performance or software availability.
- High priority: Issue prevents normal lab work on one computer.
- Critical priority: Issue affects many systems or complete lab operation.

12.5 AI Assistance Logic

In the first version, AI assistance can be implemented as rule-based recommendations. Later, machine learning can be added after enough historical data is collected.

Input Data	Possible Recommendation
Repeated disk full alerts	Clean temporary files or increase storage.
Frequent high RAM alerts	Check startup applications or upgrade memory.
Repeated offline status	Check LAN cable, switch port or network adapter.
Same software missing on many computers	Schedule software installation for the lab.
Frequent issue reports from one computer	Mark computer for detailed maintenance.

Chapter 13: Implementation Plan

13.1 Development Strategy

The project should be developed module by module. First, a small two-computer LAN prototype should be built. Then monitoring, database, dashboard, WebSocket, issue management and remote command modules can be added step by step.

13.2 Recommended Folder Structure

```
slms-project/  
  backend/  
    app/  
      main.py  
      database.py  
      models/  
      schemas/  
      routes/  
      websocket/  
      services/  
      requirements.txt  
  client-agent/  
    main.py  
    collector.py  
    communication.py  
    command_handler.py  
    ui.py  
    config.json  
  admin-dashboard/  
    src/  
      components/  
      pages/  
      services/  
      App.jsx  
    package.json  
  docs/  
    project-documentation.docx  
    api-reference.md  
  README.md
```

13.3 Development Phases

Phase	Work	Output
Phase 1	Basic LAN client-server communication	Client sends hostname/IP to server
Phase 2	System monitoring	CPU, RAM, disk and OS details visible on server
Phase 3	Database integration	Client data stored in SQLite/PostgreSQL
Phase 4	Admin dashboard	React dashboard shows computer list and status
Phase 5	WebSocket live updates	Dashboard updates without refresh
Phase 6	Software and running apps	Inventory and running app information stored
Phase 7	Issue and maintenance module	Problem reporting and maintenance history

Phase 8	Remote command module	Approved commands sent to client and logged
Phase 9	Reports and alerts	Lab summaries and threshold alerts
Phase 10	Packaging and deployment	PyInstaller client and optional Electron host

13.4 Weekly Mentor Reporting Plan

Week	Planned Work	Mentor Review Output
Week 1	Finalize title, problem statement, scope and stack	Approved project abstract and SRS outline
Week 2	Build basic Python client and FastAPI server	LAN data transfer demo
Week 3	Add psutil monitoring and database storage	System details stored successfully
Week 4	Create React dashboard layout	Computer list and status cards
Week 5	Add WebSocket live updates	Real-time CPU/RAM/disk updates
Week 6	Add software inventory and running apps	Application monitoring demo
Week 7	Add issue reporting and maintenance records	Issue management workflow
Week 8	Add remote commands with audit logs	Safe command demo and logs
Week 9	Add reports, alerts and basic AI rules	Report screenshots and recommendation demo
Week 10	Testing, bug fixing and documentation	Final project report and presentation

13.5 Sample Client Data Collection Code

```
import platform
import socket
import psutil

hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)

system_info = {
    "hostname": hostname,
    "ip_address": ip_address,
    "os_name": platform.system(),
    "os_version": platform.version(),
    "cpu_usage": psutil.cpu_percent(interval=1),
    "ram_usage": psutil.virtual_memory().percent,
    "disk_usage": psutil.disk_usage("/").percent
}

print(system_info)
```

13.6 Sample FastAPI Endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class MetricPayload(BaseModel):
    hostname: str
    ip_address: str
    cpu_usage: float
    ram_usage: float
    disk_usage: float

@app.post("/api/metrics")
def receive_metrics(payload: MetricPayload):
    # Validate and store payload in database
    return {"message": "Metrics received", "computer": payload.hostname}
```

Chapter 14: Testing and Validation

14.1 Testing Strategy

Testing verifies that each module works individually and also works together as a complete system. The system should be tested on at least two computers connected to the same LAN before expanding to all lab computers.

14.2 Test Cases

Test ID	Test Scenario	Expected Result
TC-01	Start backend server	Server starts successfully and API documentation is accessible.
TC-02	Start client agent	Client collects hostname, IP, OS and sends registration request.
TC-03	Send heartbeat	Server updates client as online and records last seen time.
TC-04	Disconnect client network	Server marks client offline after timeout.
TC-05	Upload CPU/RAM/disk metrics	Metrics are stored and visible in dashboard.
TC-06	Upload installed software list	Software inventory appears for selected computer.
TC-07	Create issue report	Issue is stored with open status and shown in issue table.
TC-08	Update issue status	Issue changes from open to in progress/resolved.
TC-09	Send approved remote command	Client receives command, executes permitted action and returns result.
TC-10	Unauthorized command request	Backend rejects request and logs attempt.
TC-11	High disk usage threshold	System generates low storage alert.
TC-12	Dashboard live update	Metric value changes without page refresh.
TC-13	Report generation	Lab status or issue report is generated correctly.
TC-14	Client packaging	PyInstaller executable runs without opening source files.

14.3 Types of Testing

Testing Type	Purpose
Unit Testing	Check individual functions such as data collection, API validation and database insert.
Integration Testing	Check complete flow from client to backend to database to dashboard.
System Testing	Verify the full system in actual lab LAN environment.
Security Testing	Check authentication, role permissions and remote command restrictions.

Performance Testing	Check update interval, dashboard response and server load with multiple clients.
User Acceptance Testing	Mentor/lab assistant verifies that system solves practical lab problems.

14.4 Validation Checklist

- All connected clients appear in dashboard.
- Offline clients are detected correctly.
- Metrics update at the configured interval.
- Database records match actual client data.
- Remote commands are logged with result status.
- Issue reports can be created, updated and resolved.
- Reports contain accurate values and dates.

Chapter 15: Security, Privacy and Reliability

15.1 Security Principles

- Only authorized users should access the admin dashboard.
- Passwords should be stored as secure hashes, not plain text.
- Remote commands must be limited to an approved whitelist.
- All command requests must be logged with user, computer, time and result.
- Client-server communication should be restricted to the lab network during prototype.
- Input data should be validated before database storage.

15.2 Role-Based Access Control

Role	Permissions
Administrator	Full dashboard access, user management, command permissions, reports and settings.
Technician	View assigned issues, update maintenance records and view computer details.
Teacher	View lab readiness and issue status; limited reporting permissions.
Client Agent	Only sends data and receives approved commands for its own computer.

15.3 Remote Command Safety

Remote command handling must be designed carefully. The system should not accept free-text operating system commands from the dashboard. It should only accept predefined command types stored in the backend. The client should verify the command type before executing it and should return the command result to the server.

15.4 Privacy Considerations

- The project should collect only technical system data required for lab management.
- Student personal data should not be collected unless the college explicitly requires it.
- Screenshots, if implemented in future, should be permission-based and clearly documented.
- Logs should be used for maintenance and security review, not unnecessary surveillance.

15.5 Reliability Measures

- Use heartbeat timeout to detect disconnected clients.
- Retry failed client-server requests after a delay.
- Store command status to avoid duplicate execution.
- Use database backups for final deployment.
- Show clear error messages when server or network is unavailable.

Chapter 16: Deployment and User Manual

16.1 Prototype Deployment Steps

1. Connect host computer and client computers to the same LAN.
2. Install Python and required packages on development systems.
3. Start FastAPI backend using Uvicorn on the host computer.
4. Configure client with host IP address and server port.
5. Run client agent on one client computer and verify registration.
6. Open admin dashboard and confirm that the client appears online.
7. Add more clients after the first client-server test succeeds.
8. Use SQLite during prototype and migrate to PostgreSQL for final deployment.

16.2 Final Deployment Plan

- Package client agent as executable using PyInstaller.
- Install client executable on all lab computers.
- Configure client auto-start according to lab policy.
- Run backend server on host/admin computer or dedicated server.
- Use PostgreSQL database for final data storage.
- Package administrator dashboard using Electron if desktop application mode is required.
- Create backup and restore procedure for the database.

16.3 Administrator User Manual

1. Open the SLMS admin dashboard.
2. Login using administrator credentials.
3. Check dashboard summary cards for total, online and offline systems.
4. Open the computer list to view system metrics and status.
5. Open a computer detail page to view software, usage, issues and command history.
6. Open the issue module to create, update or resolve problem reports.
7. Use remote commands only when required and only for approved lab management purposes.
8. Open the reports section to generate lab status, issue and maintenance reports.

16.4 Technician User Manual

1. Login with technician account.
2. Open assigned issues.
3. Check issue description, priority and computer details.
4. Perform required maintenance physically or through approved procedure.
5. Update maintenance record with action taken and result.
6. Mark issue as resolved after verification.

Chapter 17: Results and Expected Output

17.1 Expected Functional Output

Feature	Expected Output
Client registration	Client computer appears in dashboard with hostname and IP address.
Live monitoring	CPU, RAM, disk and status values update automatically.
Software inventory	Installed applications are listed for each computer.
Running applications	Currently active applications are shown with process details.
Issue report	Problem is stored, assigned and tracked until resolved.
Remote command	Approved command is sent, executed and logged with result.
Alert system	Low disk, high usage or offline systems create notifications.
Reports	Administrator can view lab summary, issue summary and maintenance summary.

17.2 Expected Benefits

- Reduces manual computer checking in the lab.
- Improves visibility of lab system health.
- Helps identify problematic systems quickly.
- Maintains proper maintenance history.
- Supports faster decision-making for lab administrators.
- Provides a strong final-year project with frontend, backend, database, networking and system programming concepts.

17.3 Screenshots to Add After Implementation

- Login screen screenshot.
- Dashboard home screenshot.
- Computer list and computer detail screenshot.
- Live metric update screenshot.
- Issue report screenshot.
- Remote command history screenshot.
- Report generation screenshot.

Chapter 18: Limitations and Future Enhancements

18.1 Current Limitations

- Initial prototype may support only a limited number of client systems.
- Some installed application details may vary depending on operating system permissions.
- Running process list may contain system processes that require filtering.
- AI module requires sufficient historical data before advanced prediction becomes useful.
- Remote command features must follow college policy and operating system permissions.
- Cloud access is not mandatory in the core version.

18.2 Future Enhancements

- Multi-lab support with lab-wise dashboard filtering.
- Advanced AI-based predictive maintenance.
- Email or mobile notifications for critical alerts.
- Role-based report export in PDF or Excel format.
- Software blocklist and unauthorized application detection.
- Scheduled maintenance calendar integration.
- Improved client auto-update mechanism.
- Optional cloud backup for reports and maintenance history.
- Mobile app for technician issue updates.
- Advanced analytics dashboard with long-term usage trends.

Chapter 19: Conclusion

Smart Lab Management System is a practical and useful final-year diploma project for managing college computer laboratories. It combines system monitoring, networking, backend development, frontend dashboard design, database management, issue tracking, remote command handling, reporting and future AI assistance. The project directly addresses real lab problems such as manual inspection, delayed fault detection, missing maintenance history and lack of centralized visibility.

The selected technology stack is suitable for the project because Python simplifies client-side monitoring, FastAPI provides a modern backend, SQLAlchemy supports database management, React supports a dynamic dashboard and WebSocket enables real-time communication. SQLite supports quick prototyping, while PostgreSQL is suitable for final deployment. PyInstaller and Electron help package the client and host parts as desktop applications.

By completing SLMS module by module, the project team can demonstrate a complete software engineering workflow from requirement analysis to design, implementation, testing and deployment. The final system can help a college lab administrator monitor all systems more effectively and maintain proper records for future maintenance.

Chapter 20: References

- Python official documentation for core language and standard libraries.
- psutil documentation for system and process monitoring concepts.
- FastAPI documentation for REST API and WebSocket development.
- Uvicorn documentation for ASGI server usage.
- SQLAlchemy documentation for ORM and database interaction.
- PostgreSQL documentation for relational database concepts.
- React documentation for user interface development.
- Vite documentation for frontend build tooling.
- Electron documentation for desktop application packaging.
- PyInstaller documentation for Python application packaging.
- Software Engineering and SRS format references used for academic project documentation.

Appendix A: Glossary

Term	Meaning
Client Agent	Software running on each lab computer to collect and send data.
Host Computer	Main computer/server used by administrator to monitor clients.
REST API	Request-response communication method between client, backend and frontend.
WebSocket	Real-time two-way communication channel.
Heartbeat	Periodic signal sent by client to show it is online.
ORM	Object Relational Mapper; used to interact with database using code models.
Audit Log	Record of important system actions and user actions.
Inventory	List of software and hardware details available in a computer.

Appendix B: Risk Analysis

Risk	Impact	Mitigation
Client not connecting to server	Dashboard does not show system	Check LAN, host IP, firewall and server status.
Antivirus blocks client executable	Client may not run	Use proper packaging, clear naming and lab administrator approval.
Too many duplicate processes	Running app list becomes unclear	Filter system processes and group duplicate names.
Database grows large	Reports become slow	Use PostgreSQL, indexing and data archiving.
Unauthorized command attempt	Security issue	Use authentication, role permissions and audit logs.
Power/network failure	Temporary disconnection	Use heartbeat timeout and reconnection logic.

Appendix C: Project Presentation Points

- Our project solves the problem of manual computer lab monitoring.
- The system has one host and many client computers connected through LAN.
- Each client sends CPU, RAM, disk, OS, IP, software and issue details to the host.
- The admin dashboard shows live status, alerts, software inventory, issues and reports.
- Remote commands are approved, permission-based and logged for safety.
- The system works without mandatory internet because it is designed for local lab network.
- Future AI module can recommend maintenance actions and detect repeated problems.